ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO THỰC TẬP

NGÀNH: CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI:

DATA ENGINEERING FOR AXIOM PLATFORM

Giảng viên hướng dẫn:	**TS. Nguyễn Thu Trang**
Sinh viên:	**Trần Minh Nam**
Mã sinh viên:	**23021646**
Lớp:	**K68I - CS4**

Hà Nội, tháng 9 năm 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành tới cô TS. Nguyễn Thu Trang và thầy TS. Nguyễn Văn Sơn đã hướng dẫn, định hướng nghiên cứu và góp ý cho em trong quá trình thực hiện đề tài. Các góp ý của thầy cô giúp em rất nhiều trong việc thực hiện và hoàn thiện quá trình nghiên cứu và báo cáo dự án thực tập này.

Em cũng xin gửi lời cảm ơn tới các thầy cô, anh chị và các bạn trong nhóm nghiên cứu Intelligent Software Engineering (iSE) đã hỗ trợ em rất nhiều về kiến thức chuyên môn trong quá trình thực hiện đề tài. Em xin cảm ơn nhóm dự án AXIOM Data Engineering, gồm các bạn Trần Mạnh Tân, Vũ Hải Đăng, Thân Đăng Khôi và chị Nguyễn Hà Linh, đã cùng em trao đổi, phản biện và xây dựng giải pháp trong suốt thời gian qua.

Cuối cùng, em xin chân thành cảm ơn các thầy cô đang công tác và giảng dạy tại Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội đã tạo điều kiện thuận lợi để em được học tập, thực tập và phát triển đề tài này.

Hà Nội, tháng 9 năm 2026

Sinh viên

Trần Minh Nam

LỜI CAM ĐOAN

Em là Trần Minh Nam, sinh viên lớp K68I-CS4, ngành Công nghệ Thông tin, Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội. Em xin cam đoan những nội dung trình bày trong báo cáo thực tập với đề tài "Data Engineering for AXIOM Platform" là kết quả công việc do em trực tiếp thực hiện trong khuôn khổ dự án AXIOM Data Engineering, dưới sự hướng dẫn của cô TS. Nguyễn Thu Trang.

Toàn bộ số liệu và kết quả thực nghiệm trong báo cáo được đo trên bộ khung đánh giá của nhóm và được trình bày trung thực. Phần công việc do các thành viên khác trong nhóm thực hiện được ghi rõ ở mục II.b.6. Các nội dung tham khảo từ tài liệu khác đều được ghi rõ nguồn trích dẫn. Em xin chịu trách nhiệm về lời cam đoan này.

Hà Nội, tháng 9 năm 2026

Sinh viên

Trần Minh Nam

MỤC LỤC

LỜI CẢM ƠN **2**
LỜI CAM ĐOAN **3**
MỤC LỤC **4**
I. Giới thiệu chung **6**
a. Giới thiệu công việc 6
b. Giới thiệu qua bài toán 6
II. Yêu cầu bài toán **7**
a. Mô tả bài toán tổng thể 7
b. Các module và phân công công việc 8
1. Data discovery 8
2. Ingestion và chuẩn hoá 8
3. Chunking và embedding 9
4. Retrieval 9
5. Tích hợp AXIOM 9
6. Phân chia công việc trong nhóm 9
III. Tóm tắt lý thuyết, giải pháp, thuật toán **10**
a. Cơ sở lý thuyết và các nghiên cứu liên quan 10
1. Bài toán retrieval và đơn vị truy xuất 10
2. Các chiến lược chunking 11
3. Hybrid retrieval 11
4. Truy xuất theo cấu trúc tài liệu và DISRetrieval 12
5. Visual document retrieval và reranking 14
6. Dữ liệu và thước đo đánh giá 15
b. Giải pháp đề xuất và kết quả thực nghiệm 16
1. Thực nghiệm module chunking và embedding 16
2. Kiến trúc thành phần retrieval 17
3. SEP, lan truyền bằng chứng theo cấu trúc 18
4. Nhánh visual retrieval 19
5. Tầng reranking 20
6. Chất lượng retrieval và chi phí vận hành 20
7. Điều kiện áp dụng của SEP 21
8. Đánh giá chất lượng câu trả lời end-to-end 22
9. Tối ưu thời gian chạy 23
c. Liên hệ và so sánh với các cách tiếp cận đã có 25
IV. Mô tả phần mềm cài đặt **25**
a. Môi trường và cách cài đặt 25
b. Cách sử dụng và tái lập kết quả 26
c. Ràng buộc môi trường thực nghiệm 26
V. Kết quả đạt được và hướng phát triển **26**

a. Kết quả đạt được 26
b. Kỹ năng và kiến thức thu thập được 27
c. Hạn chế của giải pháp hiện tại 27
d. Hướng phát triển tiếp theo 28
TÀI LIỆU THAM KHẢO **29**

I. Giới thiệu chung

a. Giới thiệu công việc

Trong kỳ thực tập, em tham gia dự án AXIOM Data Engineering thuộc nhóm Nghiên cứu và Phát triển (R&D) của nền tảng AXIOM. Mục tiêu cốt lõi của nền tảng là cho phép doanh nghiệp truy vấn thông tin bằng ngôn ngữ tự nhiên trên toàn bộ kho tài liệu nội bộ, đồng thời nhận về câu trả lời luôn đi kèm bằng chứng xác thực. Yêu cầu truy vết bằng chứng này chính là điểm quan trọng của AXIOM bởi trong môi trường doanh nghiệp, một hệ thống không thể truy rõ nguồn gốc câu trả lời sẽ không đảm bảo độ tin cậy để đưa vào vận hành thực tế.

Trong hệ thống, phần việc em phụ trách bao gồm hai module ở nửa đầu quy trình xử lý là module chunking-embedding cùng với module retrieval. Về cơ bản, module chunking-embedding sẽ chia tài liệu đã qua parsing thành các đoạn nhỏ (chunk), rồi biểu diễn mỗi đoạn bằng một vector số (embedding) để có thể tìm kiếm theo ngữ nghĩa, còn module retrieval dùng các chỉ mục mà module trước tạo ra để tìm và xếp hạng các trang chứa bằng chứng cho từng câu hỏi. Trong hai module này, retrieval là trọng tâm nghiên cứu chính của em.

Kết quả chính của em trong kỳ thực tập gồm ba phần. Phần thứ nhất là module chunking và embedding, với cấu hình chunking được chọn bằng thực nghiệm và dùng làm mặc định cho sản phẩm. Phần thứ hai là SEP, một cơ chế tận dụng cấu trúc tài liệu để xếp hạng lại các trang, kết hợp với một nhánh visual retrieval tìm kiếm trực tiếp trên ảnh trang. Phần thứ ba là cách thu hẹp phạm vi mã hoá ảnh trang (chuyển ảnh trang thành vector bằng mô hình) theo từng câu hỏi, giúp giải pháp chạy được trong thời gian thực tế. Chi tiết từng phần được trình bày ở mục III.b.

b. Giới thiệu qua bài toán

Bài toán đặt ra cho nhóm Data Engineering là với một kho tài liệu doanh nghiệp và một câu hỏi bằng ngôn ngữ tự nhiên, hệ thống cần tìm ra những trang tài liệu chứa bằng chứng để trả lời câu hỏi, đồng thời chỉ rõ mỗi bằng chứng nằm ở tệp nào và trang nào. Danh sách trang này là ngữ cảnh để AXIOM sinh câu trả lời, bởi vậy chất lượng câu trả lời phụ thuộc trực tiếp vào việc hệ thống có tìm đúng trang hay không. Trong môi trường doanh nghiệp, bài toán này gặp hai thách thức chính:

- **Sự đa dạng của dữ liệu đầu vào:** Doanh nghiệp thực tế lưu tài liệu dưới nhiều dạng như PDF có lớp văn bản, PDF scan, ảnh, slide, bảng tính, hay tài liệu chứa công thức và biểu đồ. Mỗi dạng cần một cách parsing riêng, trong khi bằng chứng có thể nằm trong văn bản, bảng hoặc hình, nên hệ thống phải tìm được bằng chứng dù nó được lưu dưới dạng nào.

- **Giới hạn về thời gian phản hồi:** Kho dữ liệu doanh nghiệp thường rất lớn, còn các bước xử lý sâu như parsing, embedding và mã hoá ảnh trang đều tốn nhiều thời gian. Do đó hệ thống không thể chờ xử lý xong toàn bộ kho rồi mới trả lời, mà phải cân bằng giữa chất lượng tìm kiếm và thời gian người dùng phải chờ.

Mô tả chi tiết bài toán, các module trong quy trình của nhóm và phần việc em phụ trách được trình bày ở mục II.

II. Yêu cầu bài toán

a. Mô tả bài toán tổng thể

Phần Data Engineering nhận đầu vào là kho dữ liệu doanh nghiệp gồm các tài liệu đa định dạng nêu ở mục I.b, cùng câu hỏi bằng ngôn ngữ tự nhiên của người dùng. Đầu ra là danh sách k trang tài liệu liên quan nhất với câu hỏi, được sắp theo thứ tự, trong đó mỗi trang ghi rõ tệp và số trang nguồn. Danh sách này là ngữ cảnh cho khâu sinh câu trả lời của AXIOM.

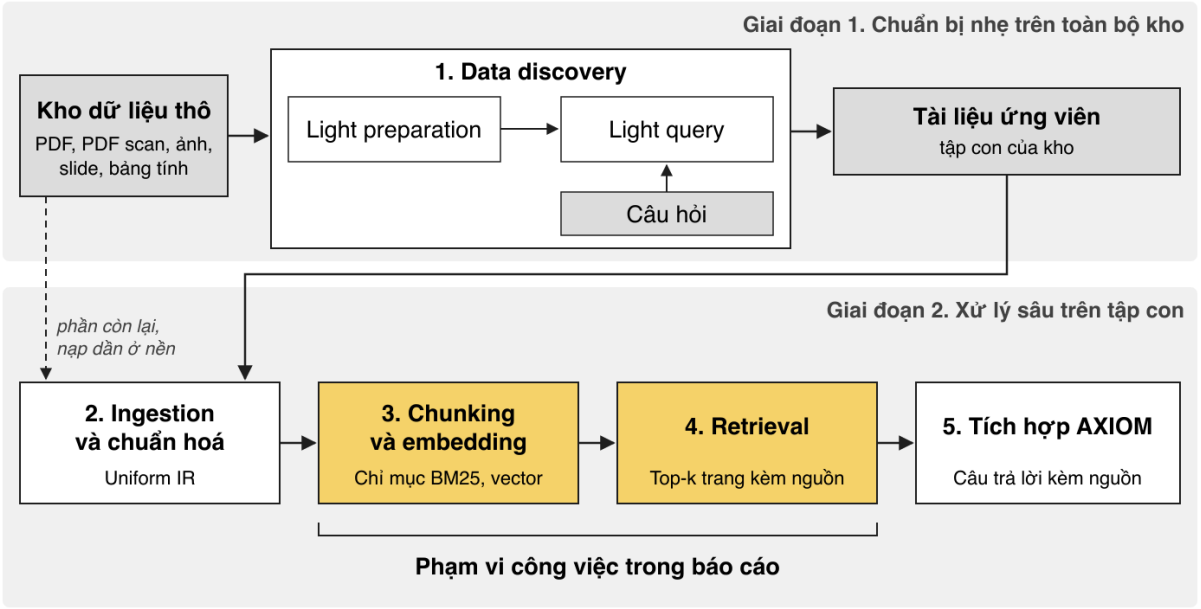
Toàn bộ quy trình phải đáp ứng bốn yêu cầu chung.

- **Yêu cầu thứ nhất là truy vết được bằng chứng:** Mọi kết quả trả về phải gắn với một tệp và một trang cụ thể.
- **Yêu cầu thứ hai là độc lập với định dạng đầu vào:** Mọi nguồn dữ liệu được chuẩn hoá về một định dạng trung gian là Uniform IR ngay tại khâu ingestion (khâu nạp và parsing dữ liệu). Nhờ vậy, khi thêm một định dạng mới, nhóm không phải sửa các module phía sau.
- **Yêu cầu thứ ba là thời gian phản hồi ở mức chấp nhận được:** Thời gian của hệ thống được xét qua ba đại lượng. Chi phí offline là thời gian chuẩn bị chạy trước một lần khi chưa có câu hỏi, gồm các bước như parsing, embedding và lập chỉ mục. *T_first* là thời gian người dùng phải chờ từ lúc đưa kho tài liệu vào hệ thống cho tới khi nhận được câu trả lời đầu tiên, còn *t_q* là thời gian online để trả lời một câu hỏi khi hệ thống đã sẵn sàng. Ba đại lượng này thường đánh đổi cho nhau, bởi dồn tác vụ nặng về phía offline giúp *t_q* nhỏ nhưng làm *T_first* tăng theo kích thước kho. Do đó các tác vụ nặng chỉ nên chạy offline khi thật cần, hoặc chỉ chạy trên tập con dữ liệu liên quan tới câu hỏi, để khi trả lời câu hỏi (online), hệ thống chỉ cần xử lý các phép tính truy xuất nhẹ và sinh câu trả lời nhanh chóng.
- **Yêu cầu thứ tư là tương thích với môi trường sản phẩm:** Các thành phần phải chạy được trên môi trường hiện có của AXIOM, gồm PyTorch 2.2.2 và

ONNX Runtime (hai thư viện chạy mô hình học sâu), với suy luận trên CPU. Các tác vụ cần GPU được tách riêng.

b. Các module và phân công công việc

Quy trình Data Engineering gồm năm module nối tiếp nhau như trong Hình 1. Mục tiêu, đầu vào, đầu ra và yêu cầu của từng module được mô tả dưới đây.



Hình 1. Quy trình xử lý dữ liệu của nền tảng AXIOM và phạm vi công việc

1. Data discovery

Module này nhận kho dữ liệu thô cùng câu hỏi của người dùng, và trả ra *Kf* tài liệu ứng viên, trong đó *Kf* là số tài liệu giữ lại cho mỗi câu hỏi. Mục tiêu của module là khoanh vùng nhanh những tài liệu có khả năng chứa câu trả lời, để nhóm không phải parsing sâu toàn bộ kho với chi phí lớn. Hệ thống trả lời trước trên tập con này, còn phần dữ liệu còn lại được nạp dần ở chế độ nền. Vì phải chạy rất nhanh, module chỉ đọc lớp văn bản sẵn có của PDF bằng pdf-inspector (thư viện đọc nhanh lớp văn bản có sẵn trong PDF) rồi lọc bằng BM25 (thuật toán xếp hạng dựa trên tần suất từ khoá), không gọi mô hình nặng nào. Module cũng phải có độ phủ cao, vì tài liệu nào bị loại ở bước này thì các module phía sau không thể tìm lại được.

2. Ingestion và chuẩn hoá

Module này nhận các tài liệu được chọn và trả ra dữ liệu có cấu trúc ở định dạng Uniform IR. Module dùng KDL (frontier parser nano, bộ công cụ parsing tài liệu thành dữ liệu có cấu trúc) để parsing sâu từng tài liệu, rồi chuẩn hoá kết quả về Uniform IR. Kết quả parsing phải giữ đúng nội dung và cấu trúc của tài liệu, gồm văn bản, đề mục, bảng và hình. Mỗi phần nội dung cũng phải mang theo tệp và số trang nguồn để kết

quả cuối cùng truy vết được. Định dạng đầu ra phải ổn định, để các module phía sau không phụ thuộc vào định dạng gốc.

3. Chunking và embedding

Module này do em phụ trách. Module nhận Uniform IR, chia nội dung thành các chunk, tính vector embedding cho từng chunk, và trả ra hai chỉ mục là chỉ mục BM25 và chỉ mục vector dùng phục vụ retrieval mà sẽ được trình bày chi tiết ở mục III.b.1.

4. Retrieval

Module này cũng do em phụ trách và là trọng tâm nghiên cứu chính. Với mỗi câu hỏi, module chấm điểm các trang trên các chỉ mục và trên ảnh trang, rồi trả ra top-k trang (*k* trang có điểm cao nhất) kèm tệp và số trang nguồn. Các trang chứa bằng chứng phải nằm ở những vị trí đầu, vì khâu sinh câu trả lời chỉ đọc top-k trang.

5. Tích hợp AXIOM

Module này đưa các thành phần đã nghiên cứu vào nền tảng AXIOM. Module tái cấu trúc quy trình cơ sở để sẵn sàng vận hành, và đóng gói thành phần retrieval thành một dịch vụ độc lập chạy trên môi trường hiện có của AXIOM.

6. Phân chia công việc trong nhóm

Dự án gồm năm thành viên. Bảng 1 tổng hợp phân công của từng người.

Bảng 1. Phân công công việc trong nhóm

Thành viên	Mã sinh viên & LHP	Mảng phụ trách
Trần Minh Nam	23021646 (K68I-CS4)	Chunking, embedding, retrieval
Trần Mạnh Tân	24021619 (K69I-CS3)	Chunking, embedding, retrieval
Vũ Hải Đăng	23020042 (K68I-IT3)	Light preparation (chuẩn bị nhanh dữ liệu cho data discovery), ingestion, chuẩn hoá Uniform IR
Thân Đăng Khôi	24020185 (K69I-IT5)	Data discovery, ingestion, quy trình xử lý theo yêu cầu của người dùng (on-demand)
Nguyễn Hà Linh	22024505 (K67I-IS)	Tích hợp vào hệ thống AXIOM

Trong mảng chunking, embedding và retrieval, em chịu trách nhiệm chính về kiến trúc và giải pháp. Bạn Trần Mạnh Tân cùng em kiểm thử các chiến lược chunking-embedding cũng như retrieval, đồng thời trực tiếp giám sát và vận hành các thực nghiệm.

III. Tóm tắt lý thuyết, giải pháp, thuật toán

a. Cơ sở lý thuyết và các nghiên cứu liên quan

1. Bài toán retrieval và đơn vị truy xuất

Truy xuất thông tin (Information Retrieval, IR) là bài toán xếp hạng một tập lớn các đơn vị nội dung theo mức độ liên quan với câu hỏi q. Cụ thể, với một tập trang tài liệu C và một câu hỏi q, cần xây dựng một hàm điểm $s_page(q, c)$ đo mức liên quan giữa câu hỏi và từng trang c, sao cho k trang chứa bằng chứng phù hợp nhất được xếp ở các vị trí đầu tiên. Trước khi xếp hạng, cần quyết định đơn vị nội dung là gì, vì lựa chọn này ảnh hưởng tới cả độ chính xác lẫn khả năng kiểm chứng bằng chứng.

- Nếu đơn vị là cả tài liệu, hệ thống dễ tìm đúng nguồn nhưng không chỉ ra được bằng chứng nằm ở đâu. Ngữ cảnh đưa cho mô hình sinh cũng quá dài và có nhiều nội dung thừa.
- Nếu đơn vị là chunk, vị trí bằng chứng được xác định rất chi tiết, nhưng mỗi chunk thường quá ngắn để chứa đủ ngữ cảnh. Một trang bị chia thành nhiều chunk rời rạc cũng làm việc xếp hạng bị nhiễu.
- Trang tài liệu cân bằng được hai cách trên, vì trang đủ dài để giữ ngữ cảnh nhưng đủ ngắn để người dùng tự kiểm tra nguồn. Trang cũng là đơn vị con người quen dùng khi trích dẫn. Vì vậy, nhóm chọn trang làm đơn vị retrieval, giống với ViDoRe V3 [1] và MMDocIR [10], hai bộ benchmark truy xuất trên tài liệu có cả văn bản lẫn hình ảnh và đều chấm điểm ở cấp trang.

Hệ thống lập chỉ mục ở cấp chunk nhưng xếp hạng ở cấp trang, nên cần một hàm quy điểm chunk về điểm trang. Nhóm dùng MaxP (Maximum Passage), trong đó điểm của một trang bằng điểm của chunk cao nhất trong trang đó.

$$s_page(q, c) = max_\{p \text{ in } c\} \; s_chunk(q, p)$$

Cách này dựa trên nhận xét rằng một trang đáng được trả về nếu nó có ít nhất một đoạn trả lời trực tiếp câu hỏi, không cần toàn bộ trang đều liên quan. Khi phân tích lỗi, báo cáo phân biệt hai loại tổn thất. Tổn thất tìm kiếm (candidate loss) xảy ra khi trang đúng không có mặt trong danh sách ứng viên ban đầu. Tổn thất xếp hạng (ranking loss) xảy ra khi trang đúng đã có trong danh sách ứng viên nhưng bị xếp thấp và rơi khỏi top-k.

2. Các chiến lược chunking

Tài liệu doanh nghiệp thường dài hơn giới hạn đầu vào của mô hình embedding, nên phải được chia thành các chunk trước khi lập chỉ mục. Cách chia quyết định ranh

giới của từng chunk, vì vậy ảnh hưởng trực tiếp tới việc mỗi chunk có giữ đủ ý hay không.

Có bốn nhóm chiến lược chunking chính.

- Fixed-size chunking cắt theo một số từ hoặc token (đơn vị từ con mà mô hình dùng để đọc văn bản) cố định, với một phần chồng lấn (overlap) giữa hai chunk liền kề. Cách này đơn giản, ổn định và gần như không tốn chi phí.
- Structural chunking cắt theo ranh giới tự nhiên của tài liệu như đoạn văn, đề mục hay bảng do parser trích xuất.
- Semantic chunking đặt điểm cắt ở chỗ độ tương đồng ngữ nghĩa giữa hai câu liên tiếp giảm mạnh.
- Hierarchical chunking dựng một cây tóm tắt nhiều cấp, điển hình là RAPTOR [12]. Cách này được mô tả chi tiết ở mục III.a.4.

Barnett và cộng sự [11] chỉ ra rằng semantic chunking thường không bù được phần chi phí tính toán và độ trễ tăng thêm. Hierarchical chunking còn tốn kém hơn vì phải gọi LLM (mô hình ngôn ngữ lớn) ngay khi lập chỉ mục. Mục III.b.1 so sánh trực tiếp fixed-size chunking và semantic chunking trên dữ liệu của nhóm.

3. Hybrid retrieval

Hybrid retrieval kết hợp hai nhánh truy xuất bổ trợ cho nhau.

- **Nhánh từ vựng dùng BM25:** BM25 chấm điểm dựa trên tần suất từ khoá trong văn bản và độ dài văn bản. BM25 khớp rất tốt các thuật ngữ hiếm, mã định danh, tên riêng và ký hiệu kỹ thuật, nhưng không hiểu từ đồng nghĩa hay các cách diễn đạt khác nhau của cùng một ý.
- N**hánh ngữ nghĩa dùng dense retrieval:** Câu hỏi và văn bản được mã hoá thành vector bằng text-embedding-3-small (mô hình embedding của OpenAI), rồi so sánh bằng độ tương đồng cosine. Nhánh này hiểu được ý của câu hỏi kể cả khi câu hỏi dùng từ khác với văn bản gốc.

Điểm BM25 và điểm cosine có thang đo rất khác nhau, nên không thể cộng trực tiếp. Hệ thống chuẩn hoá điểm theo từng truy vấn [8], tức là với mỗi truy vấn, điểm của các ứng viên trong từng nhánh được đưa về đoạn [0, 1] bằng phép chuẩn hoá min-max.

$$s_norm(c) = [s(c) - \min s(c')] / [\max s(c') - \min s(c')]$$

Sau đó, điểm của hai nhánh được cộng có trọng số với tham số alpha nằm trong đoạn [0, 1].

$$s_hybrid(c) = alpha \cdot s_dense(c) + (1 - alpha) \cdot s_bm25(c)$$

4. Truy xuất theo cấu trúc tài liệu và DISRetrieval

Các phương pháp ở trên chấm điểm từng chunk hoặc từng trang một cách độc lập. Trong khi đó, tài liệu thật có cấu trúc. Các câu trong một đoạn liên quan tới nhau, và các đoạn trong một tài liệu cùng phục vụ một chủ đề. Một số nghiên cứu gần đây tận dụng cấu trúc này bằng cách tổ chức tài liệu thành một cây.

Trước hết cần làm rõ hai khái niệm về cây. Nút lá là nút ở tầng thấp nhất, không có nút con, và chứa một đơn vị văn bản gốc như một câu hoặc một chunk. Nút trong là nút có ít nhất một nút con, và đại diện cho cả nhóm văn bản nằm bên dưới nó, gọi là cây con của nút đó. Ví dụ, một nút trong có thể đại diện cho một đoạn văn, còn các nút lá bên dưới là các câu của đoạn văn đó.

RAPTOR [12] là một cách dựng cây như vậy. RAPTOR gom các chunk có embedding gần nhau thành từng cụm (clustering), rồi dùng LLM tóm tắt mỗi cụm thành một nút trong. Quá trình này lặp lại cho tới khi chỉ còn một nút gốc. DISRetrieval [2] cũng dựng cây, nhưng dựa trên quan hệ logic giữa các câu thay vì độ giống nhau về nội dung.

Quan hệ logic này được mô tả bằng Rhetorical Structure Theory (RST), một lý thuyết ngôn ngữ học về cách các câu liên kết với nhau trong văn bản. Theo RST, mỗi nút trong ghi một quan hệ giữa các nhánh con của nó, ví dụ như đối lập, giải thích, liệt kê hay kết luận. Cây dựng theo cách này gọi là discourse tree. Ví dụ, trong một đoạn văn sáu câu, câu 1 nêu chủ đề, câu 2 và câu 3 trình bày hai ý đối lập, câu 4 và câu 5 là hai bằng chứng song song, còn câu 6 là kết luận. Khi đó, câu 2 và câu 3 được gộp dưới một nút trong mang quan hệ đối lập, câu 4 và câu 5 được gộp dưới một nút trong mang quan hệ liệt kê, và các nút này tiếp tục được gộp lên trên cho tới nút gốc. DISRetrieval gồm ba bước.

- **Bước thứ nhất là dựng discourse tree:** Nhóm tác giả huấn luyện một discourse parser, tức là một mô hình nhận vào các câu và trả ra discourse tree. Parser dựng một cây cho mỗi đoạn văn. Sau đó, bản tóm tắt của các đoạn được đưa qua cùng parser đó để dựng cây cấp tài liệu. Hai cấp được ghép thành một cây duy nhất, trong đó mỗi nút lá là một câu.
- **Bước thứ hai là gán văn bản cho các nút trong:** Sau bước thứ nhất, mỗi nút trong mới chỉ ghi một quan hệ và chưa có văn bản để so khớp với câu hỏi. Phương pháp DISRetrieval xử lý từ các nút lá lên tới gốc (bottom-up). Nếu văn bản của các nút con dài hơn một ngưỡng τ, phương pháp sẽ sử dụng mô hình

Llama-3.1-8B để tóm tắt chúng. Nếu ngắn hơn, văn bản của các nút con được nối lại. Sau đó, mọi nút đều được mã hoá thành vector embedding.

- **Bước thứ ba là truy xuất theo cấu trúc:** Hệ thống tính độ tương đồng cosine giữa vector câu hỏi và vector của mọi nút, rồi duyệt các nút theo điểm giảm dần. Nếu gặp một nút lá, câu của nút đó được chọn làm bằng chứng. Nếu gặp một nút trong, hệ thống chọn k nút lá có điểm cao nhất trong cây con của nút đó. Quá trình dừng khi đã chọn đủ K câu.

Ý tưởng cốt lõi nằm ở bước thứ ba. Khi một nút trong khớp với câu hỏi, các câu bên dưới nó được đưa vào kết quả, kể cả khi từng câu riêng lẻ chưa có điểm cao. Nhóm tác giả so sánh DISRetrieval với bốn cách làm khác.

- Flatten-chunk cắt văn bản thành các chunk tối đa 100 từ và không cắt ngang câu.
- Flatten-sentence dùng mỗi câu làm một đơn vị truy xuất.
- RAPTOR là phương pháp dựng cây bằng clustering đã nêu ở trên.
- Bisection dùng đúng bước thứ hai và bước thứ ba của DISRetrieval. Điểm khác là Bisection không dùng discourse parser mà dựng cây bằng cách chia đôi dãy câu liên tiếp thành hai nửa bằng nhau, rồi lặp lại với từng nửa. Nhờ vậy, so sánh với Bisection cho biết lợi ích của DISRetrieval đến từ quan hệ logic giữa các câu hay chỉ đến từ việc có một cây.

Các thí nghiệm được chạy trên bốn bộ dữ liệu hỏi đáp tài liệu dài, gồm QASPER (bài báo khoa học), QuALITY (bài đọc hiểu), NarrativeQA (sách và kịch bản phim) và MultiFieldQA-zh (tài liệu tiếng Trung). Trên cả bốn bộ, DISRetrieval đạt kết quả tốt nhất và Ở bước tiền xử lý, DISRetrieval nhanh hơn RAPTOR khoảng ba lần, mất 103 giây so với 338,5 giây cho một tài liệu 50.000 từ. Tuy nhiên, khi áp dụng vào bài toán của AXIOM, phương pháp vẫn có ba điểm yếu.

- **Điểm yếu thứ nhất là chi phí tiền xử lý:** Mỗi tài liệu phải qua discourse parser và nhiều lượt gọi LLM trên GPU. Một tài liệu 50.000 từ tốn khoảng 100 giây trên một GPU NVIDIA A800 80 GB. Với kho dữ liệu doanh nghiệp lớn, chi phí này làm *T_first* tăng mạnh.
- **Điểm yếu thứ hai là phụ thuộc vào discourse parser:** Nếu cây sai thì kết quả truy xuất cũng sai theo. Tuy nhiên, Discourse Parser sử dụng được huấn luyện trên RST-DT, một tập bài báo tin tức tiếng Anh đã được gán nhãn discourse tree bằng tay. Để có thể dùng cho dữ liệu tiếng Trung, nhóm tác giả phải dịch dữ liệu này bằng GPT-4o và mỗi ngôn ngữ mới sẽ lại cần phải làm như vậy.

- **Điểm yếu thứ ba là chỉ làm việc trên văn bản liền mạch**: Discourse tree được dựng từ các câu. Tài liệu doanh nghiệp lại có nhiều bảng, hình, slide và công thức, nơi cấu trúc câu không còn rõ ràng.

5. Visual document retrieval và reranking

Các kỹ thuật ở trên đều chỉ làm việc với văn bản thu được sau khi parsing. Vì vậy, nếu một thông tin không được parsing chuyển thành chữ, ví dụ số liệu trong biểu đồ hay nội dung của một sơ đồ khối, thì retrieval văn bản sẽ không bao giờ tìm thấy thông tin đó.

Visual document retrieval tránh được vấn đề này vì nó không cần parsing. Thay vì đọc chữ, mô hình nhìn trực tiếp vào ảnh của trang tài liệu, giống như cách con người lướt qua một trang để tìm thông tin. Các mô hình tiêu biểu theo hướng này là ColPali [6], ColEmbed [4] và ColVec [5]. Để so khớp câu hỏi với một trang, các mô hình này chia ảnh trang thành nhiều mảnh nhỏ và biểu diễn mỗi mảnh bằng một vector. Câu hỏi cũng được biểu diễn bằng nhiều vector, mỗi vector ứng với một token. Khi chấm điểm một trang, hệ thống lấy từng token trong câu hỏi, tìm mảnh ảnh giống với token đó nhất, rồi cộng mức giống nhau của tất cả các token lại. Phép tính này gọi là MaxSim. Cách so khớp này gọi là late interaction, vì câu hỏi và trang được mã hoá riêng và chỉ được so khớp với nhau ở bước cuối.

$$MaxSim(q, p) = \sum_{t \in q} \max_{v \in p} \langle q_t, v_v \rangle$$

Các ký hiệu trong công thức được giải thích như sau.

- *MaxSim(q, p)*: Điểm số độ tương đồng giữa truy vấn q và trang tài liệu p.
- *q*: Tập hợp các vector đại diện cho truy vấn.
- *p*: Tập hợp các vector đại diện cho nội dung hình ảnh của trang p.
- *t:* Một token (thành phần) cụ thể trong truy vấn q.
- *q_t:* Vector biểu diễn của token t.
- *v:* Một mảnh ảnh (image patch) cụ thể trong trang p.
- *v_v:* Vector biểu diễn của mảnh ảnh v.
- *<q_t, v_v>:* Tích vô hướng (dot product) giữa vector token truy vấn và vector mảnh ảnh, thể hiện mức độ khớp giữa chúng.

Điểm yếu của hướng này nằm ở chi phí. Để tìm được trang, hệ thống phải đưa từng trang qua một mô hình thị giác và ngôn ngữ có hàng tỉ tham số, nên kho càng lớn thì thời gian chuẩn bị càng dài. Một số nghiên cứu gần đây giảm chi phí này bằng cách chỉ xử lý phần thật sự cần. LightSTAR [19] dùng từ khoá để chọn trước một số trang

ứng viên rồi mới gọi mô hình nặng trên các trang đó, còn AgenticOCR [20] chỉ parsing đúng vùng trang mà câu hỏi cần tới.

Reranking là một cách khác để tăng chất lượng, thường dùng mô hình cross-encoder. Khác với nhánh dense, nơi câu hỏi và tài liệu được mã hoá riêng rồi mới so sánh (bi-encoder), cross-encoder đọc cả câu hỏi và nội dung trang cùng lúc, nhờ vậy đánh giá mức liên quan chính xác hơn. Đổi lại, cross-encoder phải chạy lại cho từng cặp câu hỏi và trang, nên chi phí tăng theo số ứng viên. Vì vậy, reranking thường chỉ được dùng cho một danh sách ngắn, ví dụ 20 trang đầu.

6. Dữ liệu và thước đo đánh giá

Đề tài dùng bộ benchmark ViDoRe V3 [1]. Thực nghiệm chính được chạy trên tập con physics bằng tiếng Pháp, gồm 302 câu hỏi trên 1.674 trang của 42 tài liệu chuyên ngành. Trung bình mỗi tài liệu dài 39,9 trang và mỗi câu hỏi có 7,2 trang đúng. Bốn tập con khác là pharmaceuticals (dược phẩm), hr (nhân sự), industrial (công nghiệp) và computer_science (khoa học máy tính) được dùng để kiểm tra SEP trên các miền dữ liệu khác (mục III.b.7).

Chất lượng xếp hạng được đo bằng NDCG@10 (Normalized Discounted Cumulative Gain tại 10) có phân cấp. Ý tưởng của thước đo này là một danh sách tốt phải đặt các trang liên quan nhất lên đầu. Trong ViDoRe V3, mỗi trang đúng được gán một mức độ liên quan. NDCG@10 cộng điểm của 10 trang đầu, nhưng trang ở vị trí càng thấp thì điểm càng bị giảm bớt. Tổng này sau đó được chia cho tổng lớn nhất có thể đạt được khi các trang được xếp theo thứ tự lý tưởng (IDCG@10), nên kết quả luôn nằm trong khoảng từ 0 đến 1.

$$NDCG@10 = DCG@10 / IDCG@10$$

$$DCG@10 = \Sigma_{i=1}^{10} (2^{rel_i} - 1) / log_2(i + 1)$$

Trong đó, rel_i là mức độ liên quan của trang ở vị trí thứ i.

Recall@10 được dùng để chẩn đoán lỗi. Recall@10 đo tỉ lệ trang đúng xuất hiện trong 10 trang đầu, nhờ vậy giúp phân biệt tổn thất tìm kiếm với tổn thất xếp hạng.

$$Recall@10 = |C_{true} \cap C_{retrieved}^{10}| / |C_{true}|$$

Trong đó, C_true là tập trang đúng của câu hỏi, còn C_retrieved^10 là 10 trang có điểm cao nhất mà hệ thống trả về.

Chi phí được đo bằng ba đại lượng đã định nghĩa ở mục II.a là chi phí offline, T_first và t_q.

Để các so sánh ở mục III.b được công bằng, mọi cấu hình retrieval đều dùng chung một kết quả parsing từ pdf-inspector và KDL, cùng 302 câu hỏi physics và cùng các thước đo trên. Nhờ đó, chênh lệch giữa các cấu hình chỉ đến từ thành phần retrieval.

b. Giải pháp đề xuất và kết quả thực nghiệm

1. Thực nghiệm module chunking và embedding

Để chọn cấu hình chunking cho module ở mục II.b.3, nhóm so sánh semantic chunking với fixed-size chunking 512 từ, overlap 128 từ trên ba bộ dữ liệu. Ngoài QASPER (mục III.a.4), hai bộ còn lại là UIT-ViQuAD, một bộ hỏi đáp đọc hiểu tiếng Việt, và ISE Challenge, kho dữ liệu đa định dạng của cuộc thi iSE Summer Challenge 2026. Kết quả được trình bày ở Bảng 2, trong đó cột thay đổi tương đối so sánh fixed-size chunking với semantic chunking trên cùng bộ dữ liệu.

Bảng 2. So sánh semantic chunking và fixed-size chunking

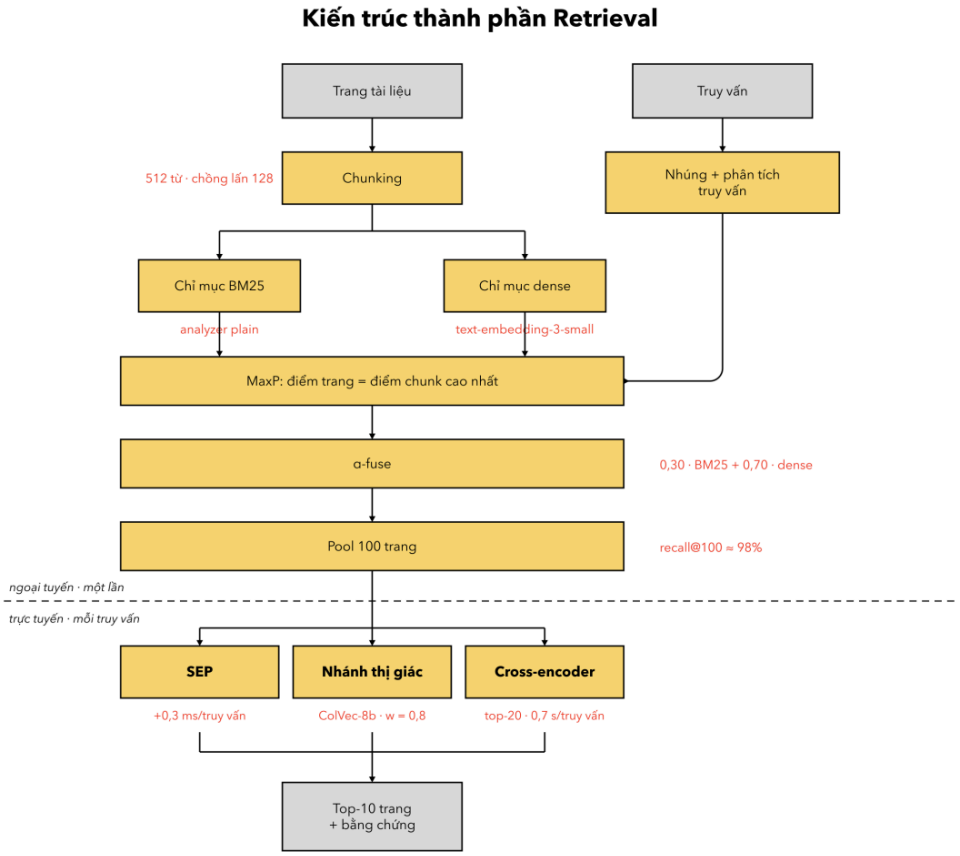
Bộ dữ liệu	Chiến lược	Recall@10	Thay đổi tương đối	Số chunk	Thay đổi số chunk
QASPER	Semantic	17,01%	—	4.115	—
QASPER	Fixed 512, overlap 128	18,85%	+10,8%	2.221	−46,0%
UIT-ViQuAD	Semantic	44,36%	—	2.183	—
UIT-ViQuAD	Fixed 512, overlap 128	46,32%	+4,4%	559	−74,4%
ISE Challenge	Semantic	61,22%	—	104.927	—
ISE Challenge	Fixed 512, overlap 128	60,21%	−1,7%	13.114	−87,5%

Trên QASPER và UIT-ViQuAD, fixed-size chunking cho recall cao hơn semantic chunking lần lượt 10,8% và 4,4% tương đối. Trên ISE Challenge, recall giảm nhẹ 1,7%, nhưng số chunk phải lưu giảm tới 87,5%. Trên cả ba bộ, số chunk giảm từ 46% đến 87,5%. Vì vậy, nhóm chọn fixed-size chunking 512 từ, overlap 128 làm cấu hình mặc định cho sản phẩm.

2. Kiến trúc thành phần retrieval

Hình 2 mô tả kiến trúc của quy trình retrieval cơ sở. Ở phía offline, mỗi trang sau khi parsing được cắt thành các chunk 512 từ, overlap 128 từ. Mỗi chunk vừa được mã hoá thành vector 1536 chiều bằng text-embedding-3-small, vừa được đưa vào chỉ mục

BM25. Ở phía online, câu hỏi cũng được mã hoá thành vector và được tách từ theo đúng cách đã áp dụng cho tài liệu, rồi được chấm điểm song song trên hai chỉ mục.



Hình 2. Kiến trúc thành phần retrieval, với ranh giới giữa chi phí offline và chi phí online

Vì đơn vị đánh giá là trang, hai danh sách chunk được quy về hai danh sách trang bằng MaxP (mục III.a.1). Hai danh sách trang sau đó được chuẩn hoá min-max theo từng truy vấn và cộng có trọng số, với 0,30 cho nhánh BM25 và 0,70 cho nhánh dense. 100 trang có điểm cao nhất tạo thành pool, tức danh sách ứng viên cho các bước sau. Mọi cơ chế cải tiến ở các mục tiếp theo đều chỉ sắp xếp lại pool này.

3. SEP, lan truyền bằng chứng theo cấu trúc

SEP (Structural Evidence Propagation) là đóng góp chính của em ở thành phần retrieval. SEP được thiết kế từ kết quả chẩn đoán lỗi của quy trình cơ sở.

Em phân loại 302 câu hỏi physics theo trạng thái của top-10 (10 trang có điểm cao nhất). Với 7,0% câu hỏi, tài liệu chứa đáp án không xuất hiện trong top-10. Với 68,9% câu hỏi, tài liệu đúng đã có trong top-10 nhưng trang đúng thì chưa. 24,2% còn lại đã có đủ trang đúng trong top-10. Trong khi đó, 98,3% câu hỏi đã có ít nhất một trang đúng trong pool 100 trang. Như vậy, vấn đề chính là tổn thất xếp hạng chứ không phải tổn thất tìm kiếm, vì hệ thống thường tìm đúng tài liệu nhưng xếp sai trang.

Em tiếp tục xét các ứng viên ở hạng 11 đến 100. Nếu ứng viên nằm cùng tài liệu với một trang đã ở top-10, xác suất nó là trang đúng là 7,83%, còn nếu nằm ở tài liệu khác thì xác suất này chỉ là 0,46%, thấp hơn khoảng 17 lần. Trong khi đó, việc nằm cạnh một trang có điểm cao chỉ làm xác suất tăng thêm 1,63 lần. Quy trình cơ sở chấm điểm từng trang độc lập nên không tận dụng được thông tin này.

SEP mượn cơ chế cốt lõi của DISRetrieval (mục III.a.4), tức là dùng một nút trong khớp với câu hỏi để nâng điểm các nút lá bên dưới nó. Điểm khác là SEP không dựng discourse tree. SEP dùng một cây hai cấp, trong đó mỗi tài liệu là một nút trong và các trang của tài liệu là các nút lá. Cây này đã có sẵn trong định danh của mỗi trang, ví dụ physics::Cours_avance_Ch-0-Presentation_Cours_Avance#page=1 gồm tên tập dữ liệu, tên tài liệu và số trang. Vì vậy SEP không cần discourse parser, không cần LLM tóm tắt và không cần đọc nội dung trang. Với mỗi trang c trong pool, điểm số được tính lại theo công thức

$$s'(c) = \lambda \cdot s(c) + (1 - \lambda) \cdot [\, \beta \cdot A_file(c) + (1 - \beta) \cdot N(c) \,]$$

Trong công thức này, s(c) là điểm tổ hợp gốc của trang sau chuẩn hoá. A_file(c) là trung bình ba điểm cao nhất của các trang cùng tài liệu có mặt trong pool. N(c) là bằng chứng từ các trang lân cận cùng tài liệu, trong phạm vi hai trang mỗi phía, với trọng số giảm theo 0,5 mũ khoảng cách. Hai tham số được đặt là $\lambda = 0,5$ và $\beta = 0,75$.

Ba lựa chọn thiết kế đều xuất phát từ số liệu chẩn đoán.

- Lựa chọn thứ nhất là dùng trung bình thay vì tổng: Nếu dùng tổng, tài liệu dài có nhiều trang trong pool sẽ luôn được lợi. Trung bình ba điểm cao nhất đo mức liên quan của tài liệu chứ không đo độ dài.
- Lựa chọn thứ hai là $\beta = 0,75$, nghiêng về tín hiệu cấp tài liệu: Tín hiệu cùng tài liệu mạnh gấp khoảng 17 lần, còn tín hiệu lân cận chỉ thêm 1,63 lần.
- Lựa chọn thứ ba là chỉ sắp xếp lại pool, không thêm ứng viên mới: Nhờ vậy recall trên pool 100 trang không đổi, và mọi thay đổi đo được đều là thay đổi về thứ tự.

Về chi phí, SEP chỉ thực hiện phép cộng có trọng số trên các điểm đã có. Phương pháp không gọi API, không cần GPU, không đổi chỉ mục và chỉ làm tăng độ trễ 0,3 ms mỗi truy vấn. SEP cần tài liệu đủ ngắn để tín hiệu cấp tài liệu có ý nghĩa. Điều kiện này được kiểm chứng ở mục III.b.7.

4. Nhánh visual retrieval

SEP sửa được lỗi xếp sai thứ tự giữa các trang trong cùng tài liệu. Để tìm nguyên nhân của phần lỗi còn lại, em chia các trang đúng thành hai nhóm, nhóm được hệ

thống tìm thấy và nhóm bị bỏ sót, rồi đo tỉ lệ ký tự đến từ các khối hình (Figure) trong kết quả parsing của mỗi nhóm.

Ở nhóm bị bỏ sót, tỉ lệ này là 0,246, gấp 2,4 lần mức 0,101 của nhóm được tìm thấy, trong khi tổng số ký tự của hai nhóm gần bằng nhau. Nói cách khác, các trang bị bỏ sót chứa nhiều thông tin ở dạng hình hơn, và phần thông tin này khó được giữ lại qua parsing. Vì vậy, em xây dựng một nhánh retrieval làm việc trực tiếp trên ảnh trang theo hướng visual document retrieval (mục III.a.5).

Mỗi trang được render thành ảnh 144 DPI thẳng từ tệp PDF, không qua OCR (nhận dạng chữ trong ảnh) và không phụ thuộc vào kết quả parsing. Ảnh trang được mã hoá bằng một trong hai mô hình late interaction, là webAI-ColVec1.1-8b [5] (8 tỉ tham số, sau đây gọi tắt là ColVec-8b) hoặc ColQwen2-2B (2 tỉ tham số). Điểm của mỗi trang được tính bằng MaxSim như ở mục III.a.5. Sau đó, điểm visual và điểm của nhánh văn bản được chuẩn hoá riêng theo từng truy vấn rồi cộng có trọng số, với trọng số 0,8 cho nhánh visual vì nhánh này cho tín hiệu mạnh hơn trên tập dữ liệu đang xét.

5. Tầng reranking

Cơ chế cải tiến thứ ba là reranking. 20 trang đầu của pool được đưa vào cross-encoder llama-nemotron-rerank-vl-1b-v2 [14], một mô hình reranking 1 tỉ tham số của NVIDIA đọc được cả văn bản và hình ảnh, để chấm điểm lại. Cơ chế này tăng chất lượng nhưng làm độ trễ online tăng thêm khoảng 0,7 giây mỗi câu hỏi.

6. Chất lượng retrieval và chi phí vận hành

Bảng 3 tổng hợp kết quả của các cấu hình theo thiết lập đánh giá ở mục III.a.6. Cột thay đổi so sánh mỗi cấu hình với quy trình cơ sở.

Bảng 3. Kết quả retrieval và chi phí vận hành trên 302 câu hỏi

Phương pháp	NDCG@10	Recall@10	Thay đổi NDCG@10	Online mỗi truy vấn	Chi phí offline
Quy trình cơ sở	43,45	46,36	—	21,7 ms	1h 23m
Kết hợp SEP	45,52	48,24	+2,08 điểm (+4,8%)	22,0 ms (+0,3 ms)	1h 23m
Kết hợp rerank Nemotron	47,42	48,95	+3,98 điểm (+9,2%)	721,7 ms (+700 ms)	1h 23m
Kết hợp visual ColQwen2-2B	47,00	49,50	+3,55 điểm (+8,2%)	22,7 ms (+1,0 ms)	1h 38m (+15m GPU)
Kết hợp SEP và ColQwen2-2B	47,73	50,71	+4,28 điểm (+9,9%)	23,0 ms (+1,3 ms)	1h 38m (+15m GPU)

Phương pháp	NDCG@10	Recall@10	Thay đổi NDCG@10	Online mỗi truy vấn	Chi phí offline
Kết hợp visual ColVec-8b	51,73	56,16	+8,28 điểm (+19,1%)	23,0 ms (+1,3 ms)	1h 58m (+35m GPU)
Kết hợp SEP và ColVec-8b	**53,15**	**55,91**	**+9,70 điểm (+22,3%)**	**23,3 ms (+1,6 ms)**	**1h 58m (+35m GPU)**

Cả ba cơ chế đều cải thiện chất lượng so với quy trình cơ sở, nhưng chi phí rất khác nhau. Cấu hình tốt nhất là SEP kết hợp nhánh visual ColVec-8b, đạt 53,15 điểm NDCG@10, tức tăng 9,70 điểm hay 22,3% so với quy trình cơ sở, trong khi độ trễ online chỉ tăng 1,6 ms.

Nhánh visual đóng góp nhiều nhất, tăng 8,28 điểm khi dùng ColVec-8b. Chi phí của nhánh này nằm gần như hoàn toàn ở phía offline, gồm khoảng 35 phút GPU để mã hoá toàn bộ ảnh trang. SEP tăng ít hơn, 2,08 điểm, nhưng gần như không tốn chi phí. Reranking tăng 3,98 điểm nhưng là cơ chế duy nhất làm độ trễ online tăng đáng kể, thêm 0,7 giây mỗi truy vấn, tức gấp hơn ba mươi lần độ trễ của cả quy trình cơ sở. Vì vậy, reranking không được đưa vào cấu hình cuối cùng.

Kết quả của hai mô hình visual cũng cho thấy giá trị của nhánh visual phụ thuộc chủ yếu vào mô hình được chọn. Với cùng cách tổ hợp, thay ColQwen2-2B bằng ColVec-8b làm mức tăng đi từ 3,55 điểm lên 8,28 điểm.

Khi so với các kết quả đã công bố, cần lưu ý cách đo khác nhau. Bảng xếp hạng MTEB (bảng xếp hạng công khai các mô hình embedding) ghi nhận ColVec-8b đạt 51,50 điểm và nemotron-colembed-vl-8b-v2 [4] đạt 50,84 điểm trên ViDoRe V3. Các con số này là trung bình trên sáu ngôn ngữ của một mô hình đơn lẻ, còn 53,15 là kết quả của cả một quy trình, đo riêng trên tiếng Pháp. Dù vậy, kết quả này cho thấy quy trình của nhóm đạt mức cạnh tranh với các giải pháp tốt hiện nay.

7. Điều kiện áp dụng của SEP

Mục III.b.3 đã lập luận rằng SEP chỉ có tác dụng khi tài liệu đủ ngắn, để tín hiệu cấp tài liệu thực sự thu hẹp được phạm vi tìm kiếm. Để kiểm tra lập luận này, em chạy SEP trên năm tập con của ViDoRe V3 có độ dài tài liệu khác nhau (Bảng 4).

Bảng 4. Thay đổi NDCG@10 khi thêm SEP trên các tập con của ViDoRe V3

Tập con	Số trang trung bình mỗi tài liệu	Tỉ lệ trang đúng trên độ dài tài liệu	Thay đổi NDCG@10	Thay đổi tương đối
physics	39,9	14,6%	+2,02	+4,7%

Tập con	Số trang trung bình mỗi tài liệu	Tỉ lệ trang đúng trên độ dài tài liệu	Thay đổi NDCG@10	Thay đổi tương đối
pharmaceuticals	44,5	12,0%	+2,17	+4,9%
hr	79,3	9,5%	+0,54	+1,0%
industrial	194,2	5,2%	−0,14	−0,3%
computer_science	680,0	0,8%	−0,09	−0,2%

SEP có tác dụng khi trang đúng chiếm khoảng 10% độ dài tài liệu trở lên. Khi tỉ lệ này giảm, A_file được tính trên một tập trang ngày càng lớn và dần tiến về mức trung bình của cả tài liệu, nên không còn giúp phân biệt trang đúng với các trang khác. Trên hai tập con có tài liệu dài nhất là industrial và computer_science, SEP còn làm chất lượng giảm nhẹ. Vì vậy, SEP chỉ nên được bật cho các kho dữ liệu gồm tài liệu ngắn.

8. Đánh giá chất lượng câu trả lời end-to-end

Mục này kiểm tra xem mức tăng của retrieval có chuyển thành câu trả lời tốt hơn hay không. Em chạy đánh giá end-to-end, tức là từ câu hỏi tới câu trả lời cuối cùng, trên toàn bộ 302 câu hỏi. Mô hình sinh là DeepSeek-V4-Flash, một mô hình nhanh và rẻ, nhận top 10 trang làm ngữ cảnh. Câu trả lời được GPT-4o chấm điểm so với đáp án (Bảng 5).

Bảng 5. Kết quả đánh giá end-to-end trên 302 câu hỏi

Cấu hình retrieval	NDCG@10	Trả lời chính xác hoàn toàn	Đúng hoàn toàn hoặc một phần
Quy trình cơ sở	43,45	50,17%	91,03%
Kết hợp SEP	45,52	51,83%	89,70%
Kết hợp SEP và ColVec-8b	53,15	50,50%	93,69%

NDCG@10 tăng 22,3%, nhưng tỉ lệ trả lời chính xác hoàn toàn gần như không đổi, chỉ tăng tương đối 0,7%, còn tỉ lệ đúng hoàn toàn hoặc một phần tăng tương đối 2,9%. Nguyên nhân là tập dữ liệu có nhiều bằng chứng trùng lặp. Mỗi câu hỏi có trung bình 7,2 trang đúng, trong khi mô hình sinh thường chỉ cần khoảng 2 trang để trả lời, nên mức cải thiện tối đa mà retrieval có thể mang lại cho kết quả end-to-end chỉ khoảng 3,6 điểm. Để tìm nút thắt thật sự, em giữ nguyên kết quả retrieval và chỉ đổi mô hình sinh từ DeepSeek-V4-Flash sang gpt-5.2, một mô hình mạnh hơn của

OpenAI. Khi đó, tỉ lệ trả lời đúng tăng 27,73 điểm, tức tăng tương đối 55,3%. Như vậy, **nút thắt của chất lượng câu trả lời nằm ở mô hình sinh chứ không nằm ở retrieval**. Việc chọn một mô hình sinh vừa nhanh vừa đủ mạnh cho sản phẩm thuộc phần việc của nhóm AXIOM, nên báo cáo này tập trung vào chunking, embedding và retrieval.

9. Tối ưu thời gian chạy

Cấu hình tốt nhất ở Bảng 3 có một điểm yếu khi vận hành. Cấu hình này cần 1 giờ 58 phút chi phí offline trước khi trả lời được câu hỏi đầu tiên, gồm 1 giờ 23 phút cho nhánh văn bản và khoảng 35 phút GPU để mã hoá toàn bộ 1.674 trang cho nhánh visual. Trên kho dữ liệu doanh nghiệp thật, chi phí này tăng tuyến tính theo số trang và nhanh chóng trở nên quá lớn. Vì nhánh visual đóng góp phần lớn chất lượng, các thực nghiệm dưới đây chỉ dùng nhánh visual và tìm cách giảm chi phí của nhánh này theo hai bước.

1)Bước thứ nhất là thu hẹp phạm vi mã hoá: Với mỗi câu hỏi, module data discovery của bạn Đăng và bạn Khôi trả về Kf tài liệu ứng viên chỉ dựa trên lớp văn bản sẵn có của PDF, với chi phí vài mili giây. Khi đó, chỉ các trang thuộc những tài liệu này mới cần mã hoá. Em quét nhiều giá trị Kf để tìm mức thu hẹp an toàn (Bảng 6).

Bảng 6. Chất lượng của nhánh visual theo số tài liệu Kf được giữ lại

Phạm vi	Độ phủ tài liệu	Số trang mỗi truy vấn	Tỉ lệ so với cả kho	NDCG@10 (ColVec-8b)
Kf = 3	86,92%	150,1	9%	50,83
Kf = 5	91,89%	247,0	15%	51,06
Kf = 10	96,36%	483,1	29%	51,89
Kf = 15	98,68%	682,5	41%	51,72
Kf = 20	99,01%	816,5	49%	51,62
Toàn bộ kho	100%	1.674	100%	51,63

Độ phủ tài liệu là tỉ lệ câu hỏi có tài liệu đúng nằm trong Kf tài liệu được chọn. Chỉ với Kf = 3, tức 9% số trang của kho, NDCG@10 đạt 50,83, gần bằng mức 51,63 khi mã hoá toàn bộ kho. Tăng độ phủ tài liệu từ 86,92% lên 99,01% làm số trang phải mã hoá tăng 5,4 lần nhưng không làm NDCG@10 tăng đáng kể.

2)Bước thứ hai là bỏ hẳn việc lập chỉ mục trước: Khi mỗi câu hỏi chỉ cần khoảng 150 trang, hệ thống có thể mã hoá các trang này ngay lúc có câu hỏi, rồi lưu kết

quả vào cache (bộ nhớ lưu các trang đã mã hoá để dùng lại) cho các câu hỏi sau. Các con số trong Bảng 7 được tính từ các hằng số đo thực tế trên GPU NVIDIA L4, gồm render 147,9 ms mỗi trang, mã hoá 1,2 giây mỗi trang, MaxSim 0,74 ms mỗi trang và nạp mô hình 77,2 giây. Em mô phỏng 302 câu hỏi chạy lần lượt qua cache, với 20 thứ tự câu hỏi ngẫu nhiên khác nhau.

Thời gian trả lời mỗi câu hỏi phụ thuộc vào trạng thái của cache, gồm ba trường hợp.

– Trường hợp cold start là khi các trang cần dùng chưa từng được mã hoá: Câu hỏi phải chịu cả chi phí render lẫn chi phí GPU. Trường hợp này quyết định thời gian tới câu trả lời đầu tiên.
– Trường hợp warm là khi các trang đã có trong cache: Câu hỏi chỉ còn chi phí tính MaxSim. Thời gian trung vị p50, tức mức mà một nửa số câu hỏi nhanh hơn, phản ánh trường hợp này.
– Trường hợp đã lập chỉ mục sẵn là khi toàn bộ kho đã được mã hoá từ trước: Mỗi câu hỏi chỉ tính MaxSim trên toàn bộ 1.674 trang.

Bảng 7. Chi phí thời gian trên 302 câu hỏi (chữ đậm là cấu hình đề xuất)

Cấu hình	Chi phí offline	Thời gian tới câu trả lời đầu tiên	Online trung bình mỗi câu	p50	p95	Truy vấn chậm nhất
Quy trình gốc, lập chỉ mục toàn kho	1h 23m	1h 23m	21,7 ms	21,7 ms	—	—
Visual, lập chỉ mục toàn kho	40m 29s	40m 29s	1,27 s	1,27 s	—	—
Visual theo yêu cầu, Kf = 3	**1m 17s**	**5m 51s**	**7,88 s**	**0,16 s**	**35,25 s**	**4m 51s**
Visual theo yêu cầu, Kf = 10	1m 17s	14m 41s	8,18 s	0,40 s	0,48 s	13m 24s

Cấu hình đề xuất là visual theo yêu cầu với Kf = 3. So với quy trình gốc, thời gian tới câu trả lời đầu tiên giảm từ 1 giờ 23 phút xuống 5 phút 51 giây, nhanh hơn khoảng 14 lần, trong khi chất lượng không khác biệt có ý nghĩa thống kê so với việc mã hoá toàn kho. Đổi lại, thời gian online trung bình tăng lên khoảng 7,9 giây mỗi câu hỏi, vì chi phí mã hoá được dời từ bước chuẩn bị sang lúc trả lời. Một số câu hỏi ở phần đuôi vẫn chậm. Ở mức p95, tức mức mà 95% số câu hỏi nhanh hơn, thời gian là 35,25 giây, và câu hỏi chậm nhất mất 4 phút 51 giây

khi các trang cần dùng chưa có trong cache. Ngoài ra, chi phí 1,2 giây mỗi trang được đo ở batch = 1, tức mã hoá từng trang một. Các kỹ thuật như batching (mã hoá nhiều trang cùng lúc) và quantization (giảm số bit dùng để lưu trọng số mô hình) chưa được áp dụng, nên tốc độ thực tế còn có thể cải thiện thêm.

c. Liên hệ và so sánh với các cách tiếp cận đã có

Như đã nêu ở mục III.a.4, RAPTOR và DISRetrieval đều phải dựng cây bằng LLM trước khi truy xuất, mất 338,5 giây và 103 giây cho một tài liệu 50.000 từ. SEP dùng cây tài liệu và trang có sẵn trong định danh, nên không cần bước tiền xử lý nào. So với DISRetrieval, SEP giữ lại cơ chế dùng nút trong để nâng điểm các nút lá, nhưng thay discourse tree bằng cây tài liệu và trang có sẵn. SEP cũng dùng được cho mọi loại tài liệu, kể cả tài liệu nhiều bảng và hình, vì SEP chỉ đọc điểm số và định danh trang. Về kết quả, SEP tăng NDCG@10 thêm 2,08 điểm so với quy trình cơ sở, và thêm 1,42 điểm khi đã có nhánh visual ColVec-8b (Bảng 3). Đổi lại, cây của SEP chỉ có hai cấp nên thô hơn discourse tree. Với tài liệu rất dài, cấp tài liệu trở nên quá rộng và tín hiệu bị pha loãng (mục III.b.7). DISRetrieval không gặp vấn đề này vì cây của nó có nhiều cấp trung gian. Điểm khác thứ hai là ngưỡng áp dụng của SEP được đo cụ thể, nên hệ thống biết khi nào nên bật SEP.

So với RAPTOR, SEP không dùng clustering. RAPTOR có thể gộp các đoạn nằm xa nhau vào cùng một nút nếu chúng giống nhau về nội dung. SEP chỉ dùng quan hệ cùng tài liệu và quan hệ nằm gần nhau, vì chẩn đoán cho thấy đây là tín hiệu mạnh nhất trên dữ liệu đang xét.

Với các mô hình visual retrieval kiểu late interaction [4, 5, 6], nhóm không dùng mô hình thị giác làm bộ truy xuất chính. Mô hình thị giác chỉ là một nhánh tổ hợp song song với nhánh văn bản. Cách này giữ được phần lớn lợi ích trong khi chi phí online gần như không đổi. Về chi phí offline, hướng thu hẹp phạm vi mã hoá ở mục III.b.9 gần với LightSTAR [19] và AgenticOCR [20], vì cả ba đều dùng một tín hiệu rẻ để khoanh vùng trước khi gọi mô hình nặng. Điểm khác là bộ lọc của báo cáo đặt ở cấp tài liệu và dùng lại module data discovery sẵn có của nhóm. Báo cáo cũng đo được ngưỡng thu hẹp an toàn thay vì chỉ đề xuất cách làm.

Cuối cùng, nhiều hệ thống thường chồng nhiều cơ chế cải tiến lên nhau. Kết quả của báo cáo cho thấy có thể bỏ bớt các thành phần tốn kém, hoặc bật từng thành phần tuỳ theo dữ liệu đầu vào, để giảm đáng kể thời gian chạy.

IV. Mô tả phần mềm cài đặt

a. Môi trường và cách cài đặt

Hệ thống được viết bằng Python 3.11 trở lên. Môi trường được quản lý bằng Conda, và các gói phụ thuộc được cài bằng pip theo lệnh sau.

```
conda env create -f environment.yml && pip install -e .
```

Các mô hình chạy trên máy cục bộ dùng PyTorch 2.2.2 và ONNX Runtime (mục II.a). Mô hình embedding văn bản và mô hình sinh được gọi qua OpenRouter, một dịch vụ cho phép gọi nhiều mô hình qua cùng một API. Khoá API của OpenRouter được khai báo trong tệp .env.

b. Cách sử dụng và tái lập kết quả

Toàn bộ thực nghiệm được chạy qua giao diện dòng lệnh (CLI). Lệnh dựng chỉ mục offline như sau.

```
python -m src.retrieval.build_artifacts \
    --run-id <RUN_ID> --analyzer auto
```

Lệnh chạy đánh giá quy trình cơ sở như sau.

```
python -m src.evaluation.run_retrieval \
    --benchmark vidore_v3 --subset physics --language french \
    --arms bm25,dense,alpha0.7 --embedder openrouter_te3s --k 10
```

Lệnh tái lập cấu hình tốt nhất, gồm SEP và ColVec-8b, như sau.

```
python research/experiments/physics_kdl_slate.py \
    --parse pdf-inspector \
    --visual-dir data/work/vidore_physics_colvec \
    --visual-name colvec --wv 0.8
```

c. Ràng buộc môi trường thực nghiệm

Việc mã hoá ảnh trang bằng ColVec-8b được chạy trên Colab và Kaggle, vì máy cục bộ không có GPU chuyên dụng.

V. Kết quả đạt được và hướng phát triển

a. Kết quả đạt được

Kết quả chính của ba phần việc em phụ trách như sau.

- Module chunking và embedding dùng cấu hình fixed-size 512 từ, overlap 128 làm mặc định cho sản phẩm. So với semantic chunking, recall tăng tương đối 4,4% đến 10,8% trên hai bộ dữ liệu và giảm 1,7% trên bộ còn lại, còn số chunk phải lưu giảm 46% đến 87,5% (mục III.b.1).
- Chất lượng retrieval tăng từ 43,45 lên 53,15 điểm NDCG@10, tức tăng 9,70 điểm hay 22,3%, trong khi độ trễ mỗi truy vấn chỉ tăng 1,6 ms (mục III.b.6).

- SEP giúp NDCG@10 tăng 2,08 điểm với chi phí 0,3 ms mỗi truy vấn và không cần GPU (mục III.b.3 và III.b.7).
- Với cấu hình chỉ dùng nhánh visual và Kf = 3, chi phí offline giảm từ 1 giờ 23 phút xuống 1 phút 17 giây, và thời gian online trung bình khoảng 7,9 giây mỗi câu hỏi. Mỗi câu hỏi chỉ cần mã hoá khoảng 9% số trang, còn NDCG@10 đạt 50,83, không khác biệt có ý nghĩa thống kê so với mức 51,63 khi mã hoá toàn kho (mục III.b.9).

Trong các kết quả trên, nhánh visual đóng góp nhiều nhất vào chất lượng, còn SEP đóng góp ít hơn nhưng gần như không tốn chi phí. Kết quả về thời gian chạy cho thấy nhánh visual không cần mã hoá toàn bộ kho mới dùng được. Thời gian online vài giây mỗi câu hỏi là chấp nhận được, vì người dùng không phải chờ lập chỉ mục toàn bộ kho dữ liệu trước khi đặt câu hỏi.

b. Kỹ năng và kiến thức thu thập được

Về kiến thức, em hiểu cách xây dựng một hệ thống retrieval hoàn chỉnh cho tài liệu doanh nghiệp. Em nắm được kiến trúc hybrid retrieval, cách quy điểm chunk về điểm trang và cơ chế late interaction trên ảnh trang. Em cũng hiểu rằng mỗi cải tiến chất lượng phải được cân nhắc cùng với chi phí chạy của nó.

Về kỹ năng, em đã xây dựng module cho một sản phẩm thật trong monorepo (một kho mã chung cho nhiều dự án) của doanh nghiệp. Em cũng học được cách thiết kế thực nghiệm, chẩn đoán lỗi từ số liệu và chạy thực nghiệm GPU trên Colab và Kaggle.

c. Hạn chế của giải pháp hiện tại

SEP chỉ hiệu quả với tài liệu ngắn. Cây của SEP chỉ có hai cấp là tài liệu và trang, nên khi tài liệu dài hàng trăm trang, cấp tài liệu quá rộng và không còn giúp phân biệt được trang đúng. Trên hai tập con industrial và computer_science, SEP còn làm NDCG@10 giảm nhẹ (Bảng 4).

Nhánh visual mang lại mức tăng lớn nhất nhưng cũng là phần khó đưa vào sản phẩm nhất. Mô hình ColVec-8b dùng giấy phép phi thương mại [5], nên cấu hình tốt nhất hiện chưa thể dùng trong sản phẩm thương mại. Chi phí mã hoá ảnh trang cũng còn cao, nhất là ở phần đuôi. Với Kf = 3, câu hỏi ở mức p95 mất 35,25 giây và câu hỏi chậm nhất mất 4 phút 51 giây. Khi kho dữ liệu lớn hơn và gồm nhiều miền, một số câu hỏi còn có tập ứng viên lên tới hơn 600 trang. Các con số thời gian chạy này cũng mới được mô phỏng từ hằng số đo trên GPU L4, chưa được đo trên hệ thống thật.

Mức tăng của retrieval cũng chưa chuyển thành câu trả lời tốt hơn. NDCG@10 tăng 22,3%, nhưng tỉ lệ trả lời chính xác hoàn toàn gần như không đổi (Bảng 5), vì chất lượng câu trả lời hiện phụ thuộc chủ yếu vào mô hình sinh.

d. Hướng phát triển tiếp theo

Để SEP dùng được cho tài liệu dài, cây của SEP cần thêm một cấp nằm giữa tài liệu và trang. Cấp này có thể là chương hoặc mục, lấy từ các đề mục mà KDL đã trích xuất.

Để đưa nhánh visual vào sản phẩm, ColVec-8b cần được thay bằng một mô hình mã nguồn mở cho phép dùng thương mại, với chất lượng gần với ColVec-8b trên cùng bộ đánh giá. Để giảm độ trễ ở phần đuôi, nhóm sẽ đặt một ngân sách trang Kp (số trang tối đa được mã hoá cho mỗi câu hỏi) và siết chặt bộ lọc tài liệu ứng viên. Cách này có thể giảm số trang phải mã hoá ở các truy vấn chậm nhất từ hơn 600 trang xuống khoảng 195 trang. Chi phí mã hoá mỗi trang cũng có thể giảm thêm nhờ batching và quantization.

Về chất lượng câu trả lời, retrieval hiện không còn là nút thắt. Bước tiếp theo của nhóm AXIOM là tìm một mô hình sinh vừa đủ nhanh cho sản phẩm, vừa đủ mạnh để tận dụng được bằng chứng đã truy xuất.

TÀI LIỆU THAM KHẢO

[1]A. Loison, Q. Macé, A. Edy, V. Xing, T. Balough, G. Moreira, B. Liu, M. Faysse, C. Hudelot, G. Viaud, "ViDoRe V3: A Comprehensive Evaluation of Retrieval Augmented Generation in Complex Real-World Scenarios," arXiv preprint arXiv:2601.08620, 2026.

[2]H. Chen, Y. Yang, Y. Li, M. Zhang, B. Hu, M. Zhang, "Beyond Chunking: Discourse-Aware Hierarchical Retrieval for Long Document Question Answering," arXiv preprint arXiv:2506.06313, 2025.

[3]X. Chen et al., "DAT: Dynamic Alpha Tuning for Adaptive Hybrid Retrieval," arXiv preprint arXiv:2503.23013, 2025.

[4]M. Xu, G. Moreira, R. Ak, R. Osmulski, Y. Babakhin, Z. Yu, B. Schifferer, E. Oldridge (NVIDIA), "Llama Nemoretriever Colembed: Top-Performing Text-Image Retrieval Model," arXiv preprint arXiv:2507.05513, 2025.

[5]webAI, "webAI-ColVec1.1: Vision-Language Document Retrieval Model," Technical Report, Model License Non-Commercial v1.0, 2025.

[6]M. Faysse et al., "ColPali: Efficient Document Retrieval with Vision Language Models," arXiv preprint arXiv:2407.01449, 2024.

[7]X. H. Lù, "BM25S: Orders of Magnitude Faster Lexical Search via Eager Sparse Scoring," arXiv preprint arXiv:2407.03618, 2024.

[8]F. Diaz, "Regularizing Query-Based Retrieval Scores," Information Retrieval, vol. 10, no. 6, pp. 531-562, 2007.

[9]G.-H. Cho, Y. Lee, K. Jeong (NAVER), "Inference-Free Multimodal Learned Sparse Retrieval for Production-Scale Visual Document Search," arXiv preprint arXiv:2605.30917, 2026.

[10]Y. Shen et al., "MMDocIR: Benchmarking Multi-Modal Retrieval for Long Documents," in Findings of EMNLP 2025, pp. 1201-1215, 2025.

[11]S. Barnett et al., "Is Semantic Chunking Worth the Computational Cost?," in Proceedings of NAACL 2025, pp. 312-325, 2025.

[12]P. Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," in ICLR 2024, 2024.

[13]L. Ouyang et al., "OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations," arXiv preprint arXiv:2412.07626, 2025.

[14]NVIDIA, "Llama-Nemotron-Rerank-VL-1B-v2 Model Card," Hugging Face, 2025.

[15]P. Tilli, M. Mesgar, "Beyond Bag-of-Patches: Learning Global Layout via Textual Supervision for Late-Interaction Visual Document Retrieval," arXiv preprint arXiv:2605.08421, 2026.

[16]A. Georgiou, "Spatially-Grounded Document Retrieval via Patch-to-Region Relevance Propagation," arXiv preprint arXiv:2512.02660, 2026.

[17]A. Abdallah, M. Abdalla, M. Ali, A. Jatowt, "Argus-Retriever: Vision-LLM Late-Interaction Retrieval with Region-Aware Query-Conditioned MoE for Visual Document Retrieval," arXiv preprint arXiv:2606.04300, 2026.

[18]H. Sun, Y. Hou, J. Guo, B. Wang, C. Yang, J. Ni, Y. Zhang, "Unveil: Unified Visual-Textual Integration and Distillation for Multi-modal Document Retrieval," arXiv preprint arXiv:2605.24530, 2026.

[19]T. Guan, H. Wang, W. Shen, X. Yang, "LightSTAR: Efficient Visual Document Retrieval via Lightweight Selection with Vision-Adaptive Refinement," arXiv preprint arXiv:2606.23539, 2026.

[20]Z. Wang, D. Ma, H. Zhong, J. Li, W. Zhang, B. Wang, C. He, "AgenticOCR: Parsing Only What You Need for Efficient Retrieval-Augmented Generation," arXiv preprint arXiv:2602.24134, 2026.

[21]H. Yao, J. Zhong, N. Lian, J. Wang, "DocMemo: Dynamic Evidence Discovery via Probabilistic Memory-Guided Retrieval for Multi-Modal Document Understanding," arXiv preprint arXiv:2608.07067, 2026.

Ý kiến đánh giá:

Điểm số: …… Điểm chữ: ……

Hà Nội, ngày …… tháng …… năm 20……

Giảng viên đánh giá

(Ký, ghi rõ họ tên)